Système informatiques – 3.somename.pdf

1. Que fait le programme suivant ?

   Ce programme crée un démon qui crée un fichier pour garder les informations STDIN/STDERR/STDOUT. (?? qqun a une mielleure explication ? hahah)

2. D'après la structure du programme et ses appels aux fonctions systèmes, comment appel-t-on ce type de programme ?

   Ce type de programme est ce qu'on appelle un Démon. Il suit en effet les 5 étapes nécessaire pour la création d'un démon :

   i. Se démoniser – fork

      Effectuer un fork.
      Laisser le parent mourir.
      Continuer les opérations avec l'enfant.
      → Pourquoi ?
      Permet que le processus grand-parent ne reste pas bloqué en attente de son enfant (i.e l'utilisateur peut continuer à utiliser le terminal)
      Permet que le parent du démn soit systemd (i.e il peut se terminer sans soucis)
      Permet que le démon ne soit pas le maître d'un groupe de processus (cf étape2)

   ii. Créer une nouvelle session et se déconnecter du terminal – setsid

      Une session est crée lors du login d'un utilisateur sur une machine et possède un identifiant SID, elle dure jusqu'à sa déconnexion. Les processus descendants font parti de la même session.

      Un processus peut créer une nouvelle session :
      - Il devient le maître de la session.
      - Il ne reçoit plus les signaux du terminal.
      - Il continue de s'exécuter même si l'utilisateur termine sa session.

        Les descripteurs STDIN/STDERR/STDOUT restent hérités du parent, il est donc préférable de les rediriger vers des **fichiers de logs** si l'on veut conserver ces informations : lignes 81 à 85.

   iii. Changer/Sécuriser son répertoire de travail – chdir/chroot

Lorsqu'un processus est lancé, son répertoire de travail correspond au répertoire à partir duquel il a été lancé → préférable de le changer pour éviter que des fichiers soient créé par le démon n'importe où et qu'on ne puisse pas démonter le répertoire de travail du démon.

Les démons sont souvent des portes d'entrée pour des attaques sur la machine. Pour éviter une partie de ces attaques une solution est de **changer la racine «/» vue par le démon**. → après ce changement la vue du démon est limité à *path,* il ne pourra plus utiliser des fichiers en dehors de *path*. (attention, tjs possible de se sortie de la prison par un autre appel à chroot).

iv. Ajuster les droits utilisateurs – setuid

Le démon a besoin de certains droit pour accomplir certaines tâches au départ (chroot etc) mais ensuite, pour des questions de sécurité, le démon ne doit pas garder ces privilèges.
RUID : real user id = la personne qui lance le programme.
EUID : effective user id = la personne dont le processus a les droits.
Pour qu'un porgramme s'éxecute avec un EUID différent de l'UID, il faut **changer le mode du fichier** avec *chmod()*.

v. Changer les droits fichiers – umask

Un processus hérite le masque de création `umask` de son parent. Il est possible que ce masque ne convienne pas pour un démon.

3. À quoi sert l'entrée "install" du makefile ?

Cette commande doit être tapée en tant que root si des droits spéciaux sont requis pour l'installation du logiciel. En gros c'est le cas si vous installez celui-ci ailleurs que dans votre home.

4. Décrivez le code et son fonctionement en détail.

somename.c

```c
#include <signal.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/sysinfo.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>

#define PATH_LOG "/var/log"
#define NAME_LOG "myownlog.log"

void exit_err(const char* str)
{
    perror(str);
    exit(EXIT_FAILURE);
}

/* =========== getNbProc ===========
Cette fonction retourne le nombre de processus du systeme,
vous ne serez pas interroge sur cette fonction
sortie: le nombre de processus actifs du systeme
note: la fonction termine le processus d'appel en cas d'erreur
   =========== getNbProc ===========*/
int getNbProc(void)
{
    struct sysinfo si;

    if(sysinfo(&si) != 0)
        exit_err("Connot get system info");

    return si.procs;
}


void handler(int sig)
{
```

/var/log est le journal de bord du système Linux. Ce fichier enregistre divers événements comme les messages d'erreurs du système, modif de la config du réseau etc.

void exit_err() est une fonction qui affiche un message d'erreur système et quitte.

void handler() qui appelle la fonction getNbProc qui donne le nombre de processus actifs du système et, si l'ouverture du fichier *logFile* - *a* Ouvre le fichier en ajout avec écriture à la fin du fichier (est créé s'il n'existe pas) – est réussi, alors il y est inscrit les processus.

```
39      static FILE* logFile;
40      int nbProc;
41
42      nbProc = getNbProc();
43      if((logFile = fopen(NAME_LOG,"a")) == NULL)
44          exit_err("Cannot open log file");
45
46      fprintf(logFile, "User call: %d\n", nbProc);
47
48      fclose(logFile);
49  }
50
51  int main(void)
52  {
53      pid_t pid;
54      int fd;
55      static FILE* logFile;
56
57      pid = fork();
58      if(pid < 0)
59          exit_err("Cannot fork");
60      else if(pid != 0)
61          return 0;
62
63      if(setsid() == -1)
64          exit_err("Cannot setsid");
65
66      if(chroot(PATH_LOG) != 0)
67          exit_err("Cannot change root");
68      if(chdir("/") != 0)
69          exit_err("Cannot change path");
70
71      umask(S_IXUSR | S_IXGRP | S_IROTH | S_IWOTH | S_IXOTH);
72
```

main :

fd : descripteur de fichier
static FILE logFile* : *static* permet de préserver la valeur même en dehors de la porté de l variable (stockée dans la partie data segment)
fork() : création d'un processus fils en dupliquant le processus appelant -> le fils hérite d'une copie de l'ensemble des descipteurs de fichiers ouverts du parent. Chaque descripteur du fils fait référence à la même description de fichier ouvert que celui du parent.

setsid() : créer une session et fixe l'ID du groupe de processus si le processus appelant n'est pas le leader de groupe.

chroot() : modifie le répertoire racine du processus appelant par celui spécifié par le chemin path. Hérité par tous les enfants. Seul un processus priviligié peut appeler *chroot()*. Utilisation :

- prison : empêche un utilisateur ou un programme de remonter dans l'arborescence et le cantonne à une nouvelle arborescence restreinte.
- changement d'environnement : permet de basculer vers un autre système linux

(autre architecture, autre distribution, autre version). Nous détaillerons ici cette technique.

Chdir() : change le répertoire courant.

Umask() : utilisé par fopen() pour créent des fichiers pour positionner les permissions d'accès initales sur les fichiers ou répertoire

fopen() : essaie d'ouvrir le fichier en *write only*

chown() : modifie le propriétaire et le groupe d'un fichier
getuid() : obtient l'identifiant de l'utilisateur ; *setuid()* : fixe
getgid() : obtient l'identifiant du groupe ;

dup2() : crée une copie du descripteur de fichier

sigaction : examine et modifie l'action associé à un signal

```
73      if((logFile = fopen(NAME_LOG,"w")) == NULL)
74          exit_err("Cannot open log file");
75      fclose(logFile);
76
77      chown(NAME_LOG, getuid(), getgid());
78
79      setuid(getuid());
80
81      if((fd = open(NAME_LOG, O_WRONLY | O_APPEND)) == -1)
82          exit_err("Cannot open log file for input / output redirection");
83      if( (dup2(fd, STDERR_FILENO) == -1) || (dup2(fd, STDOUT_FILENO) ==
        -1) )
84          exit_err("Cannot exchange file descriptors");
85      close(fd);
86
87      struct sigaction sa;
88      sa.sa_handler = handler;
89      sigfillset(&sa.sa_mask);
90      sa.sa_flags = 0;
91      sigaction(SIGUSR1, &sa, NULL);
92
```

```
93      while(1)
94          pause();
95
96      return 0;
97  }
```

makefile

```
1  all: somename.c
2          gcc somename.c -Wall -o somename
3
4  install:
5          @echo "L'installation necessite les droits root"
6          chown root ./somename
7          chmod u+s ./somename
8          @echo "Installation reussie"
```

5. Quel sera l’utilisateur, le groupe et les droits d’accès du fichier de log une fois celui-ci créé.

   S_IXUSR 00100 le propriétaire a le droit d'exécution
   S_IXGRP 00010 le groupe a le droit d'éxecution
   S_IROTH 00004 les autres ont le droit de lecture
   S_IWOTH 00002 les autres ont le droit d'écriture
   S_IXOTH 00001 les autres ont le droit d'éxecution

6. Quelle est la commande à taper depuis un terminal pour activer le handler du programme ?
   N'importe quelle touche sauf CTRL+C, CTRL+Z car on ne peut pas être modifié par un handler. (enfin je pense)